Constructors in Java

22 August 2026 08:38

What is a constructor in Java?
A constructor in Java is a block of code like the method. It is called when an instance of a class is created. At the time of calling constructor, memory for the object is allocated in the memory. It is a special type of method which is used to initialize the object.

Every time when we create an object by using **new** keyword. It calls a default constructor. If there is no constructor available in the class, in such case Java compiler provides a default constructor by default.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. Because Java compiler creates a default constructor if your class does not have any.

Rules for creating Java constructor

1. **Constructor name must be same as its class name:** A constructor must have the same name as the class. This helps the Java compiler identify the constructor and use it while creating an object.
   For example:

```
package ClassAndObjects;


/**
 * Write a description of class Person here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Person
{
    private String name, mobileNum;
    int x;
    float f;
    double d;
    //default constructor initializes the instance variables
    // with default values, Integer have 0, String will be null
    // float => 0.0f, double => 0.0
    public Person(){
        System.out.print("\nConstructor called\n");
    }

}
```

2. **A constructor must not have no explicit return type:** A constructor does not return any value, not even void. If you specify a return type, it will be treated as a normal method, not a constructor.

3. **A Java constructor cannot be abstract, static, final, or synchronized.**

(will be explained later)

Types of Java Constructors
There are two types of constructor in Java:

1. **Default constructor(No-arg Constructor):** The default constructor does not take any parameters and is used to initialize an object with default values. If no constructor is defined, Java automatically provides a default constructor.

2. **Parameterized Constructor:** The parameterized constructor accepts parameters and is used to initialize an object with a specific values at the time of object creation.

   a. **Copy Constructor:** A copy constructor is a special type of constructor that is commonly utilized to create a new object by copying the values of an existing object of the same class.

1. **Java Default Constructor**
   When a constructor does not have any parameter is known as default constructor.

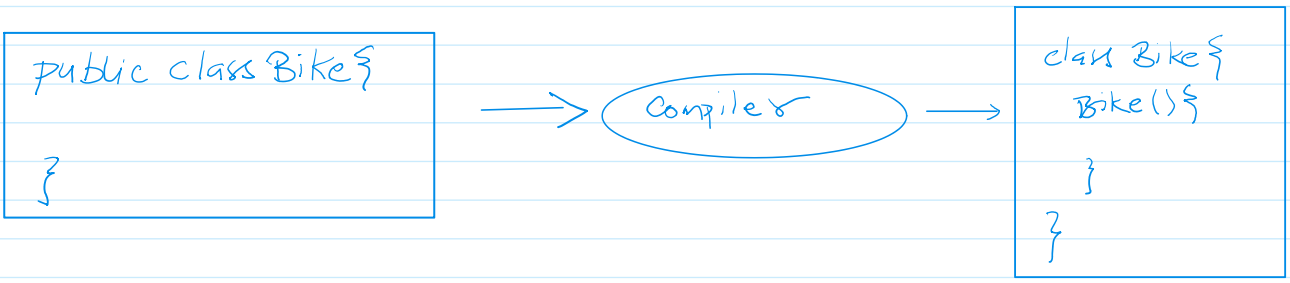
   Syntax: <class_name>(){
   //code
   }

   Example:

```
public class Bike
{
    //Creating a default constructor.
    Bike(){
        System.out.print("\nBike is created");
    }

    public static void main(String[] args){
        Bike b = new Bike();
    }
}
```

   Rule: If there is no constructor in a class, compiler automatically creates a default constructor.



Purpose of a default constructor: The default constructor is used to provide the default values to the object like zero, null etc. depending on the data type.

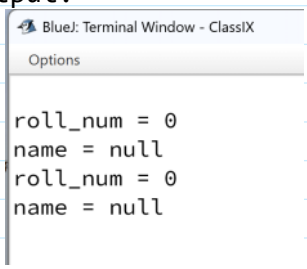
```
public class Student
{
    int roll_num;
    String name;

    void display(){
        System.out.print("\nroll_num = " + roll_num);
        System.out.print("\nname = " + name);
    }

    public static void main(String[] args){
        System.out.print("\f");
        Student s1 = new Student();
        Student s2 = new Student();

        s1.display();
        s2.display();
    }
}
```

Output:



In the above class we are not creating any constructors. So compiler provides as a default constructor. Here zero and null values are provided by default constructor.

Java parameterized constructor

A constructor that has a specific number of parameters is called a parameterized constructor.

Syntax:

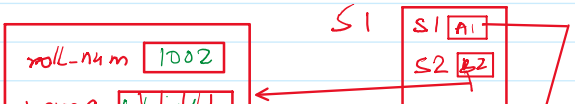
```
<class_name>(data_type parameter1, data_type parameter2, …){
   //code
}
```

Use of parameterized constructor

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructed:

```
public class Student
{
    int roll_num;
```



```
public class Student
{
    int roll_num;
    String name;

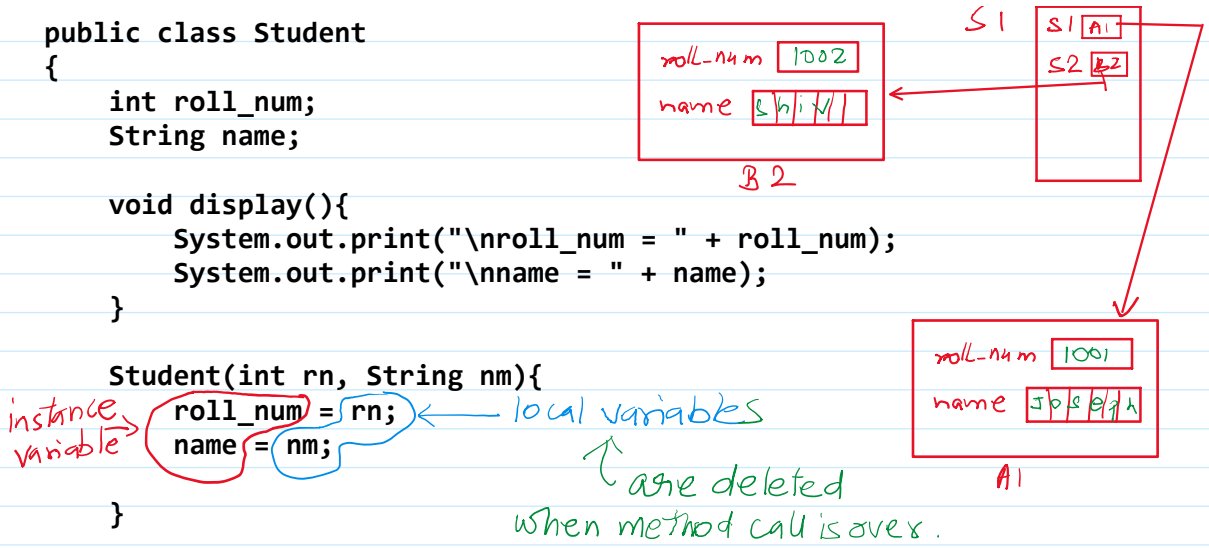
    void display(){
        System.out.print("\nroll_num = " + roll_num);
        System.out.print("\nname = " + name);
    }

    Student(int rn, String nm){
        roll_num = rn;
        name = nm;

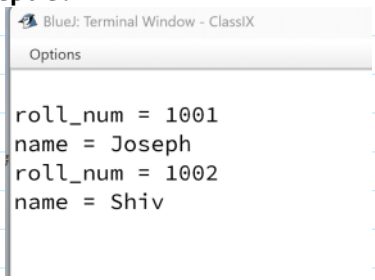
    }

    public static void main(String[] args){
        System.out.print("\f");
        Student s1 = new Student(1001, "Joseph");
        Student s2 = new Student(1002, "Shiv");

        s1.display();
        s2.display();
    }
}
```



Output:



Java Copy Constructor (Not in ICSE But in ISC)

Java does not support the copy constructor. However, we can copy the values from one object to another like a copy constructor.

```
package ClassAndObjects;


/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student
{
```

```
    int roll_num;
    String name;

    void display(){
        System.out.print("\nroll_num = " + roll_num);
        System.out.print("\nname = " + name);
    }

    Student(int rn, String nm){
        roll_num = rn;
        name = nm;
    }

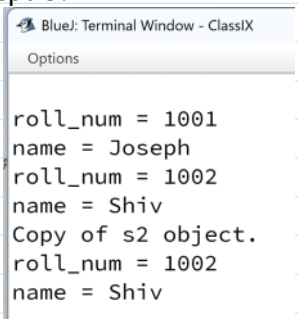
   //Copy constructor
    Student(Student s){
        roll_num = s.roll_num;
        name = s.name;
    }

    public static void main(String[] args){
        System.out.print("\f");
        Student s1 = new Student(1001, "Joseph");
        Student s2 = new Student(1002, "Shiv");

        s1.display();
        s2.display();

        Student s3 = new Student(s2); //calling copy constructor
        System.out.print("\nCopy of s2 object.");
        s3.display();
    }
}
```

Output:



Copying values without using a constructor

We can copy the values of one object into another by assigning the objects value to another object. In this case there is no need to create the constructor.

Example:

```
   public class Student
   {
       int roll_num;
       String name;
```

```
    void display(){
        System.out.print("\nroll_num = " + roll_num);
        System.out.print("\nname = " + name);
    }

    Student(){} //Default constructor
    Student(int rn, String nm){
        roll_num = rn;
        name = nm;
    }

    Student(Student s){
        roll_num = s.roll_num;
        name = s.name;
    }

    public static void main(String[] args){
        System.out.print("\f");
        Student s1 = new Student(1001, "Joseph");
        Student s2 = new Student(1002, "Shiv");

        s1.display();
        s2.display();

        Student s3 = new Student(s2);
        System.out.print("\nCopy of s2 object.");
        s3.display();

        Student s4 = new Student();
        s4.roll_num = s1.roll_num;
        s4.name = s1.name;

        System.out.print("\nCopied values without calling a constructor");
        s4.display();
    }
}
```

Constructor overloading

0 - parameter

2 - parameters

1 - parameter

Copied the values of one instance variable to another instance variable.

Output:

```
BlueJ: Terminal Window - ClassIX
Options

roll_num = 1001
name = Joseph
roll_num = 1002
name = Shiv
Copy of s2 object.
roll_num = 1002
name = Shiv
Copied values without calling a constructor
roll_num = 1001
name = Joseph
```

Constructor Overloading

In Java, a constructor is like a method but without return type. It can also be overloaded like Java methods.
Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler on the basis of the number of parameters in the list and their types.

Local variable vs instance variable

```
public class Student
{
    int roll_num;
    String name;
    int age;

    void display(){
        System.out.print("\nroll_num = " + roll_num);
        System.out.print("\nname = " + name);
        System.out.print("\nage = " + age);
    }

    Student(){} //Default constructor
    Student(int rn, String nm){
        roll_num = rn;
        name = nm;
    }

    Student(Student s){
        roll_num = s.roll_num;
        name = s.name;
    }

    Student(int roll_num, String name, int age){
        roll_num = roll_nu;
        name = name;
        age = age;
    }

    public static void main(String[] args){
        System.out.print("\f");
        Student s1 = new Student(1001, "Joseph");
        Student s2 = new Student(1002, "Shiv");

        s1.display();
        s2.display();

        Student s3 = new Student(s2);
```

When local variable name and instance variable name is same.

The local variable overshadows or dominate instance variables. Therefore, after leaving the method local variables are deleted. Hence, data is not initialized by the local variable.

The solution provided by Java is 'this' keyword. which differentiates between instance variable and local variable.

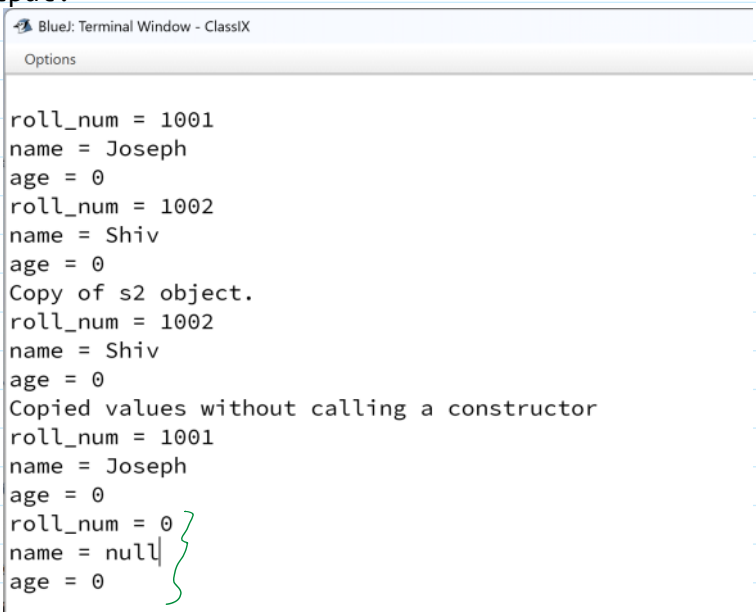
```
            System.out.print("\nCopy of s2 object.");
            s3.display();

            Student s4 = new Student();
            s4.roll_num = s1.roll_num;
            s4.name = s1.name;

            System.out.print("\nCopied values without calling a constructor");
            s4.display();

            Student s5 = new Student(1005, "Kapil", 21);
            s5.display();
        }
    }
```

Output:



After using "this" keyword

```
package ClassAndObjects;


/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student
{
    int roll_num;
    String name;
    int age;

    void display(){
        System.out.print("\nroll_num = " + roll_num);
        System.out.print("\nname = " + name);
        System.out.print("\nage = " + age);
    }
```

```
    Student(){} //Default constructor
    Student(int rn, String nm){
        roll_num = rn;
        name = nm;
    }

    Student(Student s){
        roll_num = s.roll_num;
        name = s.name;
    }

    Student(int roll_num, String name, int age){
        this.roll_num = roll_num;
        this.name = name;
        this.age = age;
    }

    public static void main(String[] args){
        System.out.print("\f");
        Student s1 = new Student(1001, "Joseph");
        Student s2 = new Student(1002, "Shiv");

        s1.display();
        s2.display();

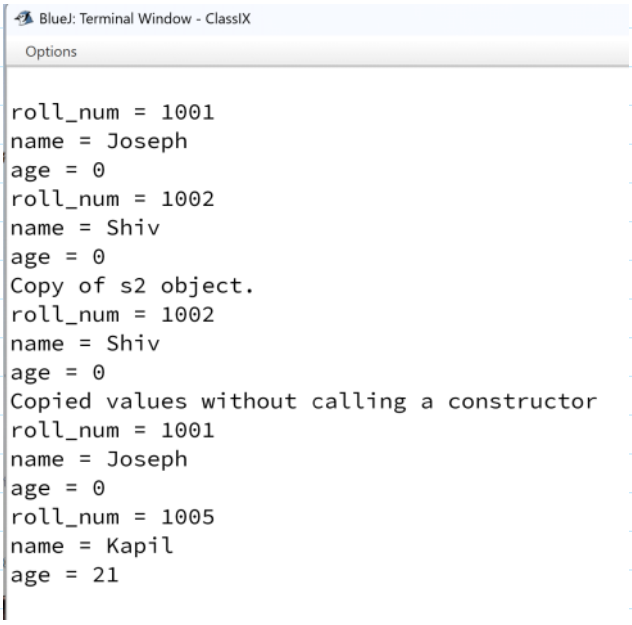
        Student s3 = new Student(s2);
        System.out.print("\nCopy of s2 object.");
        s3.display();

        Student s4 = new Student();
        s4.roll_num = s1.roll_num;
        s4.name = s1.name;

        System.out.print("\nCopied values without calling a constructor");
        s4.display();

        Student s5 = new Student(1005, "Kapil", 21);
        s5.display();
    }
}
```

Output:



Constructor Chaining

Constructor chaining in Java is a practice where one constructor calls another constructor of the same class or a super class during object creation. It is typically done using either this () to call another constructor in the same class, or super () to call a constructor in the Super class. By centralizing common construction logic, constructor chaining helps reduce code redundancy and increase code's readability.

```
package ClassAndObjects;


/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student
{
    int roll_num;
    String name;
    int age;

    void display(){
        System.out.print("\nroll_num = " + roll_num);
        System.out.print("\nname = " + name);
        System.out.print("\nage = " + age);
    }

    Student(){} //Default constructor
    Student(int rn, String nm){
        this(rn, nm, 0); //calling another constructor
    }

    Student(Student s){
        roll_num = s.roll_num;
        name = s.name;
```

```
    }

    Student(int roll_num, String name, int age){
        this.roll_num = roll_num;
        this.name = name;
        this.age = age;
    }

    public static void main(String[] args){
        System.out.print("\f");
        Student s1 = new Student(1001, "Joseph");
        Student s2 = new Student(1002, "Shiv");

        s1.display();
        s2.display();

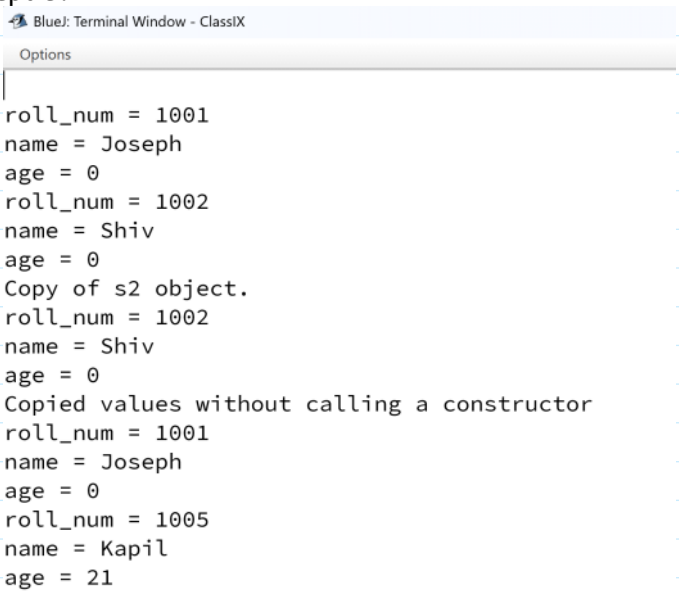
        Student s3 = new Student(s2);
        System.out.print("\nCopy of s2 object.");
        s3.display();

        Student s4 = new Student();
        s4.roll_num = s1.roll_num;
        s4.name = s1.name;

        System.out.print("\nCopied values without calling a constructor");
        s4.display();

        Student s5 = new Student(1005, "Kapil", 21);
        s5.display();
    }
}
```

Output:



Difference between constructor and a method in Java

Java Constructor	Java Method

A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly by the use of **new** keyword.	The method is invoked explicitly.
The Java compiler provides a default constructor if we do not have any constructor in a class.	The method is not provided by the compiler in any case.
The constructor's name must be same as the class name.	The method name may or may not be same as the class name.